We have analyzed the CyberPal.AI paper and created a comprehensive document that outlines the complete approach to building a cybersecurity expert LLM. Here's the short summary about the paper:

Core Approach Overview

The CyberPal.AI methodology follows a two-phase expert-driven approach:

Phase 1.1: Expert Schema-Based Generation - Create high-quality, structured instructions from cybersecurity datasets

Phase 1.2: Hybrid Structured LLM-Augmentd Synthetic Expansion - Expand and diversify the dataset using AI-powered generation

Phase 2: CONTENT-GROUND SYNTHETIC DATA GENERATION: -

Data Preparation Phase 1.1: -

The first step in preparing an instruction tuning dataset is leveraging **structured domain knowledge** from authoritative sources. This approach prioritizes **expert-defined schemas**, **structure-driven parsing**, and **relationship extraction** from cybersecurity datasets like MITRE ATT&CK, CWE, CVE, and CAPEC.

Rather than relying on an LLM to generate instructions directly from raw documents (which risks hallucinations and shallow comprehension), this method uses **custom schemas and rule-based parsing** to extract meaningful instruction templates from the data.

Goals

1. Utilize the structured nature of cybersecurity data to build diverse and grounded instruction templates.
2. Develop schemas per dataset to extract entity-level and relationship-level understanding.
3. Generate a high-quality initial instruction dataset (~150K examples) for further synthetic expansion.

Step-by-Step Breakdown

Step 1: Identify Structured Security Datasets

Dataset	Description
MITRE ATT&CK	Tactics, techniques, mitigations, detections, malware, groups
CWE	Weakness types and properties
CVE	Publicly known vulnerabilities

Dataset	Description
CAPEC	Attack patterns and how adversaries exploit weaknesses
Others	Security wikis, threat reports, interviews, Q&A forums, Sigma rules, etc.

Step 2: Schema Design

Each dataset has a unique structure, so cyberpal AI created **schemas** for parsing them into instruction templates.

A schema includes:

- **Entity types**: e.g., `Technique`, `Mitigation`, `Weakness`
- **Attributes**: `id`, `description`, `severity`, `related entities`, etc.
- **Template rules**: How to phrase instructions using the extracted data

Key Advantages of This Method

- **Schema-driven**: Ensures data consistency and task diversity
- **Human-in-the-loop**: Schemas encode expert knowledge, not random prompts
- **Zero hallucination**: Uses only grounded relationships found in structured data
- **Supports CoT**: Enables step-by-step reasoning based on known entity links

Phase 1.2: - Second Generation Step - Structured LLM-Augmented Instruction Generation

The second phase focuses on expanding the instruction dataset using **LLM-augmented reasoning**. Cyberpal combine schema structure and expert-curated paths with the reasoning power of teacher models. This enhances complexity, diversity, and cross-framework relationships.

Cyberpal apply this method to structured datasets like **BRON**, **SIEM Rules**, and **Sigma Rules**.

1. BRON Graph-Based Instruction Generation

BRON is a massive knowledge graph interlinking many security datasets (MITRE ATT&CK, CAPEC, CWE, CVE, D3FEND, etc.). It contains:

- Hundreds of thousands of nodes

- Millions of relationships

BRON allowed to generate instructions on the relationships between different objects across frameworks.

Process: -

- Graph processed and traversed into paths.
- Enriched with domain knowledge from different resources.
- Generate the reasoning ow the nodes are connected to each other and include explanation in the instruction dataset.

Key Instruction Types from BRON:

- **Direct Link Reasoning**: Use LLM to explain relationships between two connected nodes (e.g., CWE → CVE)
- **Path Reasoning (CoT)**: Generate step-by-step reasoning across paths (e.g., CWE → CAPEC → ATT&CK Technique)
- **Multi-path Reasoning**: Combine multiple paths to solve layered security tasks (e.g., From a weakness, find attack + how to detect/mitigate)

Process:

1. **Path Extraction**: Traverse BRON and collect direct/indirect node pairs.
2. **Explain Direct Edges**: Use LLM to generate relationship explanations.
3. **Build Chain-of-Thoughts**: Use node descriptions to explain each hop.
4. **Negative Sampling**: Add unlinked node pairs to train model to reject false connections.

Turn this reasoning into the predefined schema-based instruction dataset.

2. SIEM Rules to TTP Mapping

SIEM rules monitor threat signals and log events. Each rule includes metadata like:

- Rule ID, logic, pattern
- Associated MITRE tactic and technique

Goal:

Train LLM to reason about how and why a rule maps to a given tactic/technique.

Process:

1. Extract rule description + mapped MITRE technique/tactic
2. Retrieve MITRE metadata (description, usage, etc.)
3. Guide teacher LLM to reason about the mapping
4. Validate output via evaluator model
5. Final instruction: "Given Rule X, explain why it maps to Technique Y"

3. Sigma Rule Reasoning Tasks

Sigma rules are structured templates for detection logic. Each rule contains:

- `logsource` (log type)
- `detection` field (event filters)
- `level` (severity)

Instruction Types:

- **Explain how rule detects an attack**
- **Reason which attack type is detected by this rule**
- **Generate rule given attack + indicators**

Process:

1. Parse fields (logsource, detection pattern, level)
2. Feed them to LLM with a structured prompt
3. Evaluate for correctness with second LLM

Additional dataset: - Q&A, threat reports on various security threats, security and reverse-engineering stack-exchange, and Wikipedia pages related to computer security. For each dataset, define a schema based on its structure and build instructions in a similar manner to the previously mentioned datasets.

Phase 2: - Content grounded Generation: -

Two methods of synthetic data generation: -

Evol-Instruct starts with an initial set of instructions and rewrites them step by step into more complex instructions. More specifically, Evol-Instruct uses In-depth and In-breath evolving. The In-depth Evolving includes five types of operations: adding constraints, deepening, concretizing, increasing reasoning steps, and complicating input. The In-breadth Evolving is a mutation, i.e.,

generating a completely new instruction based on the given instruction.

Self-Instruct also starts with an initial set of instructions. Then, the model is prompted to generate instructions for new tasks. It leverages the existing collection of instructions to create more broad coverage instructions that define, often new, tasks. We can think of In-breadth Evolving as a variant of Self-Instruct.

Problem:- Free form generation may lead to the Hallucination and the information's those may be irrelevant or can cause accuracy issue.

Content Grounded Generation: - fused the two methods with content-grounded generation, meaning cyberpal tried to force the model to generate a more complex instruction that is grounded by the document from which the previous instruction was generated. Also used LLM as evaluator to evaluate the response.

Trained models: - used Llama-3 instruct 8B, Mistral instruct 7B v0.3, and Phi-3-medium-4k-instruct.

Eval Metrics: -
HELM: - Multiple choice QA
ROUGE: - Summarization Task.
F1: - For classification Tasks.

Xalt's Proposed Line of Work (MVP): Instruction Fine-Tuning Framework Based on CyberPal AI

Based on our analysis of the CyberPal AI methodology and their two-phase instruction-tuning pipeline (SecKnowledge Phase 1 + Phase 2), we can adopt a similar strategy for developing a domain-specialized instruction-tuned LLM. Our approach would proceed in the following phases:

Phase 1: Structured Instruction Dataset Generation (First Generation)

We will need to build a high-quality seed dataset using structured sources and expert-defined schema templates, as CyberPal did.

Steps to Take:

1. **Source Identification & Parsing**:
    - Explore and extract data from structured sources such as:

        - MITRE ATT&CK (techniques, tactics, mitigations)
        - CWE, CVE, CAPEC
        - Domain-specific detection rules (Sigma, SIEM)
        - Security Wiki, Q&A forums (if applicable)
2. **Schema and Template Development**:
    - Define instruction schemas based on entity types and relationship mappings (e.g., Technique → Tactic, CWE → CAPEC).
    - **Brainstorm templates with domain experts** to convert structured entities into instruction-answer pairs.
    - Support both:
        - **Single-entity tasks** (e.g., Describe this technique)
        - **Cross-entity reasoning** (e.g., How does this rule relate to this tactic?)
3. **Instruction Set Construction**:
    - Programmatically generate instruction-answer pairs using structured data and templates.
    - Enrich complex instructions with reasoning steps (Chain-of-Thought).

TBDs (To Be Discussed):

- What domain-specific structured datasets are available to us?
- Do we have availability of the ready to start repositories or we will need to find them or we will have to start from scratch??
- Who will serve as the domain expert for schema design and validation and instruction dataset generation rule setup.

Phase 2: LLM-Augmented Synthetic Instruction Expansion (Second Generation)

Scale generated dataset in phase 1 using teacher LLMs (of our choice paper mentioned Mixtral model) to generate deeper reasoning examples and multi-hop instructions.

Steps to Take:

1. **Dataset Selection for LLM Enrichment**:
    - Target complex datasets like:
        - BRON-style graphs (cross-framework linkages)
        - Sigma and SIEM rules (rule-to-TTP mapping)
2. **Synthetic Instruction Generation via LLMs**:
    - Use teacher models to:
        - Explain entity relationships
        - Walk graph paths
        - Expand existing instructions with more complexity (e.g., multi-step reasoning or reverse reasoning)
        - Add adversarial/negative examples for contrastive learning
3. **Evaluation and Filtering**:

- Validate the LLM-generated outputs using evaluator models to reduce hallucinations.
- Filter outputs based on reasoning quality and factual correctness.

TBDs:

- Should the synthetic data focus more on in-depth reasoning or in-breadth task variation?
- What teacher LLM and evaluator LLM infrastructure do we plan to use?
- What are our budget constrains in terms of the hardware?

Phase 3: Instruction-Fine-Tuning & Evaluation

Once dataset construction is complete, we proceed to model fine-tuning and evaluation.

Steps to Take:

1. **Model Selection**:
   - Choose an open-source base model (e.g., LLaMA, Mistral, etc. or small LM models (llama 8B, phi, qwen suited to resource constraints.
2. **Fine-Tuning Strategy**:
   - Use curriculum-style tuning:
     - Start with simpler instructions → gradually incorporate multi-hop, multi-task instructions.
   - Apply instruction-tuning with LoRA or full fine-tuning, depending on compute budget.
3. **Evaluation Benchmark Construction**:
   - Build a test suite mirroring SecKnowledge-Eval:
     - Classification, Q&A, Chain-of-Thought, and instruction-following tasks.
   - Include adversarial distractors and held-out entity types.

TBDs:

- What accuracy/latency trade-offs are acceptable for the final model?
- Do we want to support retrieval-augmented generation (RAG) or just instruction tuning?
- How will human review and quality assessment be handled for the training dataset?

Constraints Noted from CyberPal.AI Implementation

- **No Open Source Code or Dataset**:
  As of now, CyberPal's "SecKnowledge" dataset and model codebase are **not publicly available**. This means:
  - We must reconstruct the entire data pipeline from scratch

- We need to independently define schemas, templates, and evaluation sets since we do not have direct access to their expert curated template and complete methodology.